



DAY 29: REGULAR EXPRESSIONS & PATTERN MATCHING IN JAVASCRIPT

Regular Expressions (Regex) are powerful tools for pattern matching and text manipulation in JavaScript. They help with searching, validating, and replacing text efficiently. 🚀

◆ What is a Regular Expression?

A Regular Expression (Regex) is a sequence of characters that defines a search pattern.

- ◆ Used for validating input (e.g., email, phone numbers)
- ◆ Helps in searching & replacing text
- ◆ Supports complex pattern matching

You can create Regex in JavaScript using:

- 1 Literal notation: `/pattern/flags`
- 2 RegExp constructor: `new RegExp("pattern", "flags")`

Example:

```
let regex1 = /hello/; // Literal  
notation  
let regex2 = new RegExp("hello");  
// RegExp constructor
```

◆ RegEx Flags

Flags modify the behavior of pattern matching:

Flag	Description
g	Global search (find all matches)
i	Case-insensitive search
m	Multiline matching

Example:

```
let text = "Hello hello";  
let pattern = /hello/i;  
// Case-insensitive match  
console.log(text.match(pattern));  
// Output: ["Hello"]
```

◆ Common RegEx Patterns

Pattern	Meaning
^	Start of a string
\$	End of a string
.	Any single character
\d	Any digit (0-9)

<code>\w</code>	Any word character (a-z, A-Z, 0-9, _)
<code>\W</code>	Any non-word character
<code>\s</code>	Any whitespace character
<code>\S</code>	Any non-whitespace character
<code>*</code>	Match 0 or more occurrences
<code>+</code>	Match 1 or more occurrences
<code>?</code>	Match 0 or 1 occurrence
<code>{n}</code>	Match exactly n times
<code>{n,}</code>	Match at least n times
<code>{n,m}</code>	Match between n and m times

Example: Matching a phone number (format: 123-456-7890)

```
let phonePattern = /^\\d{3}-\\d{3}-\\d{4}$/;  
console.log(phonePattern.test("123-456-7890")); // true  
console.log(phonePattern.test("12345"));  
// false
```

◆ Methods for Working with RegEx

1 test() – Checks if a pattern exists

Returns true or false.

```
let pattern = /hello/i;  
console.log(pattern.test("Hello World"));  
// true
```

2 match() – Returns matching text

Returns an array of matches or null if no match is found.

```
let text = "The quick brown fox jumps  
over the lazy dog.";
let matches = text.match(/\w+/g);
// Match all words
console.log(matches);
```

3 replace() – Replaces matched text

Replaces a pattern with another string.

```
let text = "I love JavaScript!";
let newText = text.replace(/JavaScript/,  
"React");
console.log(newText); // "I love React!"
```

4 split() – Splits a string based on a pattern

```
let sentence = "apple, banana, cherry";  
let words = sentence.split(/,\s*/);  
console.log(words);  
// ["apple", "banana", "cherry"]
```

◆ Real-World Examples

◆ 1. Validate Email Address

```
let emailPattern = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;  
console.log(emailPattern.test("test@example.com")); // true  
console.log(emailPattern.test("invalid-email")); // false
```

◆ 2. Extract Numbers from a String

```
let str = "Order ID: 12345, Price: $99";  
let numbers = str.match(/\d+/g);  
console.log(numbers); // ["12345", "99"]
```

◆ 3. Remove Extra Spaces

```
let sentence = "  Hello    World  ";  
let trimmed = sentence.replace(/\s+/g, "  
").trim();  
console.log(trimmed); // "Hello World"
```